

Dissenyar un sistema orientat a objectes que modela la gestió de vehicles blindats de guerra (tancs) i els seus components, com ara armament, municions i motors.

Es crearà un nou projecte amb el nom:
CognomNomExamenUF4

En cas de no respectar el format del nom del projecte, la puntuació de l'examen serà 0 punts.

Es crearà una classe principal que anomenarem Examen2425.

Aquesta classe tindrà el mètode estàtic:

```
public static String header(String fillchar ,int total_size,String string) {  
    String result = string;  
    for (int i= 0; i < (total_size - string.length())/2 ; i++) {  
        result = fillchar+result+fillchar;  
    }  
    return result;  
}
```

Aquest mètode ens servirà per crear capçaleres. Per exemple, si anomenem el mètode com segueix:

```
System.out.println(header("=",60,"Title"));
```

Ens imprimirà:

```
=====Title=====
```

Sempre que necessitem crear una capçalera daquest tipus, haurem d'utilitzar aquest mètode de la classe principal. En qualsevol altre cas es restarà un punt a l'examen.

Les classes seran les següents:

Ammunition
CannonProjectile
MachineGunBullet

Weapon
CannonLauncher
MachineGun

Engine

Tank
LightTank
HeavyTank

I la interfície:

ArmorPiercer

PierceArmor tindrà un únic mètode **pierceArmor(Tank t)** Que servirà per deixar l'armadura del tanc t a un 30% del seu valor.

Les classes:

Ammunition:

Serà una classe que no podrem instanciar.

Tindrà els atributs **String description; int numberOfShots; int cost; float caliber;**

El constructor **Ammunition(String description, int numberOfShots, float caliber, int cost)** i definirà un mètode **getData()** que mostrarà els atributs d'aquesta classe de la següent forma:

Si (description; numberOfShots; cost; caliber) = ("Armour Piercing 120mm", 30, 120, 15000);

Armour Piercing 120mm 30 120,00 15000

Es a dir, farà servir el següent format:

`"%-40s%10d%10.2f%10d", description, numberOfShots, caliber, cost`

També implementarà els mètodes:

float getCaliber()

int getCost()

I no tindrà cap mètode més.

CannonProjectile

Heretarà d'**Ammunition**

Tindrà el constructor **CannonProjectile(String description, int numberOfShots, float caliber, int cost)**

I no tindrà cap mètode més.

MachineGunBullet

Heretarà d'**Ammunition**

Tindrà el constructor **MachineGunBullet(String description, int numberOfShots, float caliber, int cost)**

I no tindrà cap mètode més.

Weapon

Serà una classe que no podrem instanciar.

Tindrà els atributs **int cost;String description;float caliber;**

El constructor **Weapon(String description, float caliber,int cost)**

i definirà un mètode **getData()** que mostrarà els atributs d'aquesta classe de la següent forma:

Si (description, caliber,cost) = ("Cannon Royal Ordnance L7", 150f,250000);

Cannon Royal Ordnance L7 150,00 250000

Es a dir, farà servir el següent format:

`"%-50s%10.2f%10d", description, caliber, cost`

També implementarà els mètodes:

float getCaliber()

int getCost()

I no tindrà cap mètode més.

CannonLauncher

Heretarà de **Weapon**

Tindrà el constructor **CannonLauncher(String description, float caliber, int cost)**

I no tindrà cap mètode més.

MachineGun

Heretarà de **Weapon**

Tindrà el constructor **MachineGun(String description, float caliber, int cost)**

I no tindrà cap mètode més.

Engine

Tindrà els atributs int power; int cost; String description;

El constructor **Engine(String description, int power, int cost)**

i definirà un mètode **getData()** que mostrarà els atributs d'aquesta classe de la següent forma:

Si (description, power,cost) = ("Combustion Engine SVC234", 560, 9600);

Combustion Engine SVC234	560	9600
--------------------------	-----	------

Es a dir, farà servir el següent format:

"%-50s%10d%10d", description, power ,cost

També implementarà els mètodes:

int getCost()

I no tindrà cap mètode més.

Tank

Serà una classe que no podrem instanciar.

Tindrà els atributs int armor; int cost; int speed;Engine engine;CannonLauncher cannon_launcher;
String description;

El constructor **Tank(String description, int armor, int speed, Engine engine, int cost)**

El mètode int getCost(), que ens tornarà el cost total de tots els elements assemblats. És a dir, cost del propi tank + cost del motor + cost del canó ensamblat.

Els mètodes int getArmor() ; int getSpeed(), setArmor(int armor); CannonLauncher
getCannon_launcher() ; setCannon_launcher(CannonLauncher cannon_launcher).

Implementarà la interfície **Comparable** que ens permetrà ordenar els objectes de la classe **tank** en funció del cost total.

Els mètodes abstractes:

void **addMachineAmmunition(MachineGunBullet mgb);**

```

void removeMachineAmmunition(int position);
void addCannonProjectile(CannonProjectile cp);
void removeCannonProjectile(int position);
void showAmmunitionStats();

```

i definirà un mètode **toString()** que mostrarà els atributs d'aquesta classe. Una sortida d'exemple serà:

```

=====TankPanzer VI Tiger=====
Description                Armor    Speed    Cost
Panzer VI Tiger            5500    30      850000
-----Engine-----
Description                Power    Cost
Combustion Engine SVC234    560     9600
..

```

Per a les capçaleres caldrà utilitzar la funció estàtica **header** descrita amb anterioritat.

LightTank

Heretarà de **Tank**.

Tindrà els atributs `MachineGun machine_gun;` `CannonProjectile[] cannon_projectile;` `MachineGunBullet[] machine_gun_bullet;`

Tindrà el constructor **LightTank(String description, int armor, int speed, Engine engine, int cost)**

Els arrays `cannon_projectile` i `machine_gun_bullet` només tindran espai per acoblar 5 unitats de `CannonProjectile` i `MachineGunBullet` respectivament. Això s'haurà de configurar al constructor.

Els mètodes:

void setMachine_gun(MachineGun machine_gun)

void addMachineAmmunition(MachineGunBullet mgb)

Aquest mètode comprovarà primer si hi ha acoblada una metralladora; en cas contrari, mostrarà en missatge:

"First Assemble a Machine Gun" i finalitzarà.

En cas de tenir una metralladora acoblada, comprovarà que el calibre de la munició coincideix, en cas contrari mostrarà el missatge:

"Calibers does not match" i finalitzarà.

També comprovarà que tenim espai per acoblar la munició a l'array de municions, en cas contrari mostrarà el missatge:

"There no free slots to add more machine gun bullets" i finalitzarà.

Si no s'ha mostrat cap dels missatges anteriors, s'acoblarà la munició a l'array de municions de metralladora.

void removeMachineAmmunition(int position)

Aquest mètode comprovarà primer que l'enter passat és un índex vàlid de l'array, i que tenim munició en aquest índex. En cas contrari, mostrarà el missatge per consola:

"There is not Ammunition in this Slot" i finalitzarà.

En cas de passar un index correcte on hi ha munició, posarà el valor null en aquesta posició.

void addCannonProjectile(CannonProjectile cp)

Aquest mètode comprovarà primer si hi ha acoblat una canó; en cas contrari, mostrarà en missatge: *"First Assemble a Cannon Launcher"* i finalitzarà.

En cas de tenir un canó acoblada, comprovarà que el calibre de la munició coincideix, en cas contrari mostrarà el missatge:

"Calibers does not match" i finalitzarà.

També comprovarà que tenim espai per acoblar la munició a l'array de municions, en cas contrari mostrarà el missatge:

"There no free slots to add more canon projectils" i finalitzarà.

Si no s'ha mostrat cap dels missatges anteriors, s'acoblarà la munició a l'array de municions de canó.

void removeCannonProjectile(int position)

Aquest mètode comprovarà primer que l'enter passat és un índex vàlid de l'array, i que tenim munició en aquest índex. En cas contrari, mostrarà el missatge per consola:

"There is not Ammunition in this Slot" i finalitzarà.

En cas de passar un index correcte on hi ha munició, posarà el valor null en aquesta posició.

Int getCost()

Ens calcularà el cost total del tank, amb absolutament tots els elements ensamblats (tank, motor, armes, municions de les armes)

String toString()

Si creem una instància com la següent:

```
LightTank lt3 = new LightTank("Vickers Mk. VI", 1400, 40, new Engine("Gas Engine V6", 400, 8000), 130000);
lt3.setCannon_launcher(new CannonLauncher("Cannon Royal Ordnance L7", 150f,250000));
lt3.setMachine_gun(new MachineGun("DShK", 12.7f, 37000));
lt3.addCannonProjectile(new CannonProjectile("High Explosive 150mm", 20, 150, 50000));
lt3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));
```

Ens mostrarà el següent resultat:

```
=====TankVickers Mk. VI=====
Description      Armor    Speed    Cost
Vickers Mk. VI   1400     40      130000
-----Engine-----
Description      Power    Cost
Gas Engine V6    400     8000
-----Weapons-----
Description      Caliber    Cost
Cannon Royal Ordnance L7  150,00    250000
DShK             12,70     37000
-----Ammunition-----
Description      Amount    Caliber    Cost
High Explosive 150mm    20      150,00    50000
MG 12.7mm NATO BMG    600     12,70     2300
-----Total Cost-----
                                     477300
```

HeavyTank

Heretarà de **Tank** i implementarà **ArmorPiercer**

Tindrà els atributs `MachineGun[] machine_gun`; `CannonProjectile[] cannon_projectile`;

`MachineGunBullet[] machine_gun_bullet`;

Tindrà el constructor **HeavyTank(String description, int armor, int speed, Engine engine, int cost)**

Els arrays `cannon_projectile` i `machine_gun_bullet` només tindran espai per acoblar 7 i 10 unitats de `CannonProjectile` i `MachineGunBullet` respectivament. Això s'haurà de configurar al constructor.

L'array `machine_gun` tindrà capacitat per acoblar 3 `MachineGun`. Això s'haurà de configurar al constructor.

Els mètodes:

`void addMachine_gun(MachineGun machine_gun)`

Aquest mètode comprovarà que tenim espai per afegir una nova `MachineGun`. En cas contrari mostrarà el missatge:

"There is not free slots to add a new Machine Gun" i finalitzarà.

En cas de tenir espai per afegir una de nova, la afegirà a la primera posició lliure de l'array.

`void addMachineAmmunition(MachineGunBullet mgb)`

Aquest mètode comprovarà primer si hi ha acoblada com a mínim una metralladora; en cas contrari, mostrarà en missatge:

"First Assemble a Machine Gun" i finalitzarà.

En cas de tenir alguna metralladora acoblada, comprovarà que el calibre de la munició coincideix amb alguna de les metralladores acoblades, en cas contrari mostrarà el missatge:

"Calibers does not match" i finalitzarà.

També comprovarà que tenim espai per acoblar la munició a l'array de municions, en cas contrari mostrarà el missatge:

"There no free slots to add more machine gun bullets" i finalitzarà.

Si no s'ha mostrat cap dels missatges anteriors, s'acoblarà la munició a l'array de municions de metralladora.

`void removeMachineAmmunition(int position)`

Aquest mètode comprovarà primer que l'enter passat és un índex vàlid de l'array, i que tenim munició en aquest índex. En cas contrari, mostrarà el missatge per consola:

"There is not Ammunition in this Slot" i finalitzarà.

En cas de passar un índex correcte on hi ha munició, posarà el valor null en aquesta posició.

`void addCannonProjectile(CannonProjectile cp)`

Aquest mètode comprovarà primer si hi ha acoblat una canó; en cas contrari, mostrarà en missatge:

"First Assemble a Cannon Launcher" i finalitzarà.

En cas de tenir un canó acoblada, comprovarà que el calibre de la munició coincideix, en cas contrari mostrarà el missatge:

"Calibers does not match" i finalitzarà.

També comprovarà que tenim espai per acoblar la munició a l'array de municions, en cas contrari mostrarà el missatge:

"There no free slots to add more canon projectils" i finalitzarà.

Si no s'ha mostrat cap dels missatges anteriors, s'acoblarà la munició a l'array de municions de canó.

`void removeCannonProjectile(int position)`

Aquest mètode comprovarà primer que l'enter passat és un índex vàlid de l'array, i que tenim munició en aquest índex. En cas contrari, mostrarà el missatge per consola:

"There is not Ammunition in this Slot" i finalitzarà.

En cas de passar un índex correcte on hi ha munició, posarà el valor null en aquesta posició.

`Int getCost()`

Ens calcularà el cost total del tank, amb absolutament tots els elements ensamblats (tank, motor, armes, municions de les armes)

`String toString()`

Si creem una instància com la següent:

```
HeavyTank ht3 = new HeavyTank("M26 Pershing",4900,38,new Engine("Combustion Engine SVC234", 560, 9600),750000);
    ht3.setCannon_launcher(new CannonLauncher("Cannon 2A46", 140f,175000));
    ht3.addMachine_gun(new MachineGun("DShK", 12.7f, 37000));
    ht3.addMachine_gun(new MachineGun("GRANDE .50", 21.3f, 57000));
    ht3.addCannonProjectile(new CannonProjectile("Armour Piercing Composite Rigid 140mm", 30, 140, 35000));
    ht3.addCannonProjectile(new CannonProjectile("Armour Piercing Composite Rigid 140mm", 30, 140, 35000));
    ht3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));
    ht3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));
    ht3.addMachineAmmunition(new MachineGunBullet("KPV 21.3mm NATO BMG", 200, 21.3f, 8400));
    ht3.addMachineAmmunition(new MachineGunBullet("KPV 21.3mm NATO BMG", 200, 21.3f, 8400));
```

Ens mostrarà el següent resultat:

=====TankM26 Pershing=====			
Description	Armor	Speed	Cost
M26 Pershing	4900	38	750000
-----Engine-----			
Description		Power	Cost
Combustion Engine SVC234		560	9600
-----Weapons-----			
Description		Caliber	Cost
Cannon 2A46		140,00	175000
DShK		12,70	37000
GRANDE .50		21,30	57000
-----Ammunition-----			
Description	Amount	Caliber	Cost
Armour Piercing Composite Rigid 140mm	30	140,00	35000
Armour Piercing Composite Rigid 140mm	30	140,00	35000
MG 12.7mm NATO BMG	600	12,70	2300
MG 12.7mm NATO BMG	600	12,70	2300
KPV 21.3mm NATO BMG	200	21,30	8400
KPV 21.3mm NATO BMG	200	21,30	8400
-----Total Cost-----			
			934600

A la classe principal:

Es crearà un array list de tanks **ArrayList<Tank> tanks = new ArrayList<Tank>()**, on hi afegirem almenys 6 Tanks. I mostrarem els Tanks de l'array list primer ordenades per cost total i després per velocitat. per a aquest darrer cas s'haurà de fer ús de la interfície **Comparator**.

Es deixa com a exemple les instàncies següents perquè puguin ser utilitzades a l'examen.

```
LightTank lt1 = new LightTank("M24 Chaffee", 1000, 56, new Engine("Electric Engine X2", 600, 13000), 150000);
LightTank lt2 = new LightTank("T-70", 1200, 45, new Engine("Gas Engine V6", 400, 8000), 160000);
LightTank lt3 = new LightTank("Vickers Mk. VI", 1400, 40, new Engine("Gas Engine V6", 400, 8000), 130000);

lt3.setCannon_launcher(new CannonLauncher("Cannon Royal Ordnance L7", 150f,250000));
lt3.setMachine_gun(new MachineGun("DSHK", 12.7f, 37000));
lt3.addCannonProjectile(new CannonProjectile("High Explosive 150mm", 20, 150, 50000));
lt3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));

HeavyTank ht1 = new HeavyTank("Panzer VI Tiger",5500,30,new Engine("Combustion Engine SVC234", 560,
9600),850000);
HeavyTank ht2 = new HeavyTank("T-34/85",4700,35,new Engine("Combustion Engine SVC234", 560, 9600),730000);
HeavyTank ht3 = new HeavyTank("M26 Pershing",4900,38,new Engine("Combustion Engine SVC234", 560,
9600),750000);
ht3.setCannon_launcher(new CannonLauncher("Cannon 2A46", 140f,175000));
ht3.addMachine_gun(new MachineGun("DSHK", 12.7f, 37000));
ht3.addMachine_gun(new MachineGun("GRANDE .50", 21.3f, 57000));
ht3.addCannonProjectile(new CannonProjectile("Armour Piercing Composite Rigid 140mm", 30, 140, 35000));
ht3.addCannonProjectile(new CannonProjectile("Armour Piercing Composite Rigid 140mm", 30, 140, 35000));
ht3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));
ht3.addMachineAmmunition(new MachineGunBullet("MG 12.7mm NATO BMG", 600, 12.7f, 2300));
ht3.addMachineAmmunition(new MachineGunBullet("KPV 21.3mm NATO BMG", 200, 21.3f, 8400));
ht3.addMachineAmmunition(new MachineGunBullet("KPV 21.3mm NATO BMG", 200, 21.3f, 8400));
```

Sistema de Puntuació: La prova tindrà un 10 si no s'aplica cap penalització de les esmentades a continuació:

Atributs no privats **-0,5 punts** cadascun.

Formats mostrats per consola no exactes al que es demana **-0,75 punts** a cada cas.

No implementació ordenar per cost total **-1,75 punts**

No implementació ordenar per velocitat **-1,75 punts**

Cada mètode que no funcioni correctament **-0,75 punts**

Classe que hauria de ser abstracta i no ho és **-1 punt**

Herència implementada incorrectament **-1 punt**

No implementar el que es demana a la classe principal **-2 punts**

Cada classe no implementada **-1,5 punts**

En cas de no saber utilitzar el mètode estàtic header de la classe principal **-1 punt**